

Python für Roboter

Skript zu Lernbereich 3: Einen Roboter mit einer Hochsprache programmieren

fischertechnik Robotics TXT 4.0 · ROBO Pro Coding · Python 3
mit interaktiven Lernseiten (Python im Browser + TXT-Simulator)
und den Fischer-Projekten A1–A8 (Base Set)

Name: Klasse:

So arbeitest du mit diesem Skript

Jedes Kapitel erklärt ein Python-Thema am Beispiel eines Fischer-Projekts. Die grauen Kästen enthalten Code – tippe ihn ab, statt ihn nur zu lesen.

Zu jedem Kapitel gibt es eine **Lernseite** (HTML) mit Aufgaben, die du sofort im Browser ausführen und automatisch prüfen lassen kannst.

Danach baust du das Modell nach der Fischer-Bauanleitung und überträgst den Code auf den **echten TXT 4.0**. Die Lösungen zu „Teste dich“ stehen im Anhang.

Inhalt

1 Der TXT 4.0, ROBO Pro Coding und Python.....	3
2 Python-Grundlagen: Variablen, Datentypen, Rechnen.....	5
3 Vom Schaltplan zum Code: Bedingungen.....	7
4 Schleifen und Zustandsautomaten.....	9
5 Funktionen, Zeitsteuerung und Threads.....	11
6 Binärcode und Listen.....	13
7 Texte, Wörterbücher und Verschlüsselung.....	15
8 Motoren, Encoder und Kalibrierung.....	17
9 Sensorgesteuertes Fahren: Hindernisse.....	19
10 Kartesische Koordinaten: der Malroboter.....	20
11 3D-Koordinaten.....	22
12 Spurfolger und Regelung.....	24
13 Künstliche Intelligenz I: Maschinelles Lernen.....	26
14 Künstliche Intelligenz II: Neuronale Netze und Bilderkennung.....	28
Anhang A Python-Spickzettel.....	30
Anhang B TXT-4.0-Befehle (ROBO Pro Coding).....	30
Anhang C Fehlermeldungen verstehen.....	31
Anhang D Lösungen zu „Teste dich“.....	32

Ordner der Unterrichtsreihe	Inhalt
02_Praesentation	Begleitpräsentation zur Reihe (PowerPoint)
03_Arbeitsblaetter	16 Arbeitsblätter mit Lösungen
04_Skript	dieses Skript
05_HTML_Lernseiten	index.html öffnen → 14 Lektionen mit Python im Browser und TXT-Simulator
06_TXT_Programme	Python-Vorlagen für den echten TXT 4.0 (A1–A8, KI)
Material/Projekte Fischer	Aufgaben, Lösungen und Lernziele der Fischer-Projekte A1–A8

1 Der TXT 4.0, ROBO Pro Coding und Python

Lernziele dieses Kapitels

- die Bestandteile eines Robotersystems benennen
- Blockly- und Python-Programmierung vergleichen
- ein Python-Projekt in ROBO Pro Coding anlegen

1.1 Das Robotersystem

Ein Roboter arbeitet nach dem **EVA-Prinzip**: **E**ingabe über Sensoren (Taster, Fototransistor, Ultraschall, Kamera), **V**erarbeitung im Controller (TXT 4.0 mit Programm) und **A**usgabe über Aktoren (Motoren, LEDs, Servos, Display).

Anschluss	Bedeutung	Beispiele
I1 ... I8	Universal-Eingänge (digital und analog)	Taster, NTC, Fototransistor, Ultraschall, IR-Spursensor
C1 ... C4	schnelle Zählereingänge	Encoder der Motoren, Impulstaster
M1 ... M4 / O1 ... O8	Motor-Ausgänge bzw. Einzelausgänge	Encodermotoren, LEDs
S1 ... S3	Servo-Ausgänge	Greifer, Stifthalter
USB1	USB-Anschluss	Kamera

1.2 Von Blöcken zu Text

In **Blockly** steckst du Bausteine zusammen. ROBO Pro Coding übersetzt jeden Baustein in eine Zeile **Python**. Python ist eine **Hochsprache**: Der Code ist für Menschen lesbar und wird erst beim Ausführen in Maschinenbefehle (Binärcode) übersetzt. Vorteile von Text: schneller zu schreiben, leicht zu kopieren, zu kommentieren und zu vergleichen – und es gibt keine Grenzen durch fehlende Bausteine.

Blockly	Python
setze wert auf 5	<code>wert = 5</code>
warte 1 s	<code>time.sleep(1)</code>
dauerhaft wiederholen	<code>while True:</code>
wiederhole 5-mal	<code>for i in range(5):</code>
falls ... mache ... sonst	<code>if ...: ... else:</code>
definiere Unterprogramm	<code>def name():</code>

1.3 Ein Python-Projekt anlegen

- ROBO Pro Coding starten, Verbindung zum TXT herstellen (USB, WLAN oder Bluetooth).
- Neues Projekt anlegen. Unter **Konfiguration** die Bauteile an die richtigen Anschlüsse ziehen – genau wie im Schaltplan der Fischer-Aufgabe.
- ROBO Pro Coding erzeugt daraus die Datei `lib/controller.py` mit Namen wie `TXT_M_I1_mini_switch` (TXT_M = Haupt-Controller, I1 = Eingang 1, mini_switch = Bauteil).
- Tipp zum Umstieg: Programm in Blockly bauen und in die **Python-Ansicht** wechseln. So siehst du den passenden Python-Code und die genauen Befehlsnamen.
- Das Hauptprogramm steht in `main.py`. Mit  wird es auf den TXT übertragen und gestartet.

```
import time                                # Werkzeugkasten "Zeit"
from lib.controller import *               # alle konfigurierten Bauteile
from lib.display import *                  # Display des TXT

# --- Hauptprogramm ---
while True:
    widerstand = TXT_M_I1_ntc_resistor.get_resistance()
    display.set_attr("txt_label.text", str(widerstand) + " Ohm")
    time.sleep(1)
```

Wichtig

Die Befehle in diesem Skript entsprechen dem Code, den ROBO Pro Coding erzeugt. Prüfe bei Abweichungen immer die Python-Ansicht deiner Version. Im ISB-Material 3.1/3.2 kommen teilweise andere Bibliotheksnamen vor (z. B. `ftrobot2`, `motor_left.get_distance()`) – auf dem TXT mit ROBO Pro Coding gelten die Namen aus `lib/controller.py`.

Teste dich

1. Ordne zu: Ultraschallsensor, LED, Encodermotor – Eingang oder Ausgang?
2. Was bedeutet der Name `TXT_M_03_led`?
3. Warum nennt man Python eine Hochsprache?

2 Python-Grundlagen: Variablen, Datentypen, Rechnen

Lernziele dieses Kapitels

- Kommentare und Variablen sinnvoll einsetzen
- Datentypen unterscheiden und umwandeln
- Sensorwerte mit Formeln umrechnen (Fischer A1)

2.1 Kommentare und Variablen

Hinter **#** steht ein **Kommentar** – der Computer ignoriert ihn. Kommentare erklären, gliedern und schalten Zeilen beim Testen ab („auskommentieren“). Eine **Variable** ist eine beschriftete Kiste für einen Wert. = heißt „lege hinein“. Namen: klein, ohne Leerzeichen und Umlaute, sprechend (`temp_max`).

2.2 Datentypen

Typ	Beispiel	Verwendung
int	1532	ganze Zahlen: Ohm, Impulse, Geschwindigkeit 0 ... 512
float	21.4	Kommazahlen (mit Punkt!)
str	"21.4 °C"	Texte für Display und Konsole
bool	True, False	Wahrheitswerte, z. B. Taster gedrückt

Umwandeln: `str(21.4)`, `int("5")`, `float("2.5")`, `round(21.437, 1)`. Bequemer: der **f-String** `f"{temp:.1f} °C"` setzt Werte direkt in Text ein.

2.3 Rechnen

Zeichen	Bedeutung	Beispiel → Ergebnis
+ - * /	Grundrechenarten	<code>7 / 2</code> → 3.5
//	ganzzahlige Division	<code>7 // 2</code> → 3
%	Rest (Modulo)	<code>7 % 2</code> → 1
**	Potenz	<code>2 ** 10</code> → 1024
<code>math.sqrt</code> , <code>math.log</code>	Wurzel, Logarithmus (nach <code>import math</code>)	<code>math.sqrt(16)</code> → 4.0

2.4 Beispiel: Thermometer (Fischer A1)

Der TXT misst am NTC (Heißleiter) eigentlich den **Widerstand**. Die Temperatur berechnet man mit der Steinhart-Hart-Gleichung $1/T = a + b \cdot \ln(R) + c \cdot \ln(R)^3$ (T in Kelvin). Die Koeffizienten stammen aus dem Fischer-Lösungsblatt.

```

import math, time
from lib.controller import *
from lib.display import *

a, b, c = 0.004164866085, -0.0002763825278, 0.000003094580198
kleinste, groesste = 1000, -1000

while True:
    R = TXT_M_I1_ntc_resistor.get_resistance()
    T = 1 / (a + b * math.log(R) + c * math.log(R) ** 3) - 273.15
    kleinste = min(kleinste, T)
    groesste = max(groesste, T)
    display.set_attr("txt_label.text", f"{T:.1f} °C (min {kleinste:.1f} / max
{groesste:.1f})")
    time.sleep(1)

```

Merke

Sensoren liefern **Rohwerte** (Ohm, Impulse, Millivolt). Erst eine Formel oder eine Kalibrierung macht daraus eine sinnvolle Größe. Der A/D-Wandler des TXT übersetzt die analoge Spannung in eine Dualzahl.

Ausprobieren im Browser

Lernseite **L01 Von Blockly zu Python, L02 Rechnen mit Sensorwerten** (Ordner 05_HTML_Lernseiten).
Aufgaben: Widerstand messen, Abstand anzeigen, Min/Max, Steinhart-Hart.

Am echten TXT 4.0

Fischer-Aufgabe A1 „Analoge Sensoren“: Programmieraufgaben 1–3 und Experimentieraufgabe 1.
Arbeitsblätter 1 und 2.

Teste dich

1. Welchen Datentyp haben 512, 0.751 und "Hallo"?
2. Was ergibt 17 % 5?
3. Warum startet `kleinste` mit dem Wert 1000?

3 Vom Schaltplan zum Code: Bedingungen

Lernziele dieses Kapitels

- aus einem Schaltplan die Konfiguration und die Python-Befehle ableiten
- Bedingungen mit if/elif/else formulieren
- logische Operatoren and, or, not anwenden

3.1 Schaltung → Programmcode

Der Schaltplan zeigt, **welches Bauteil an welchem Anschluss** hängt. Daraus folgt die Konfiguration – und daraus der Befehl. Ein Taster ist ein **digitaler** Sensor (gedrückt = 1 = **True**), ein NTC ein **analoger** (stufenloser Wert).

Schaltplan	Konfiguration	Python-Befehl	Ergebnis
Taster an I1 (Schließer)	Mini-Taster I1	<code>TXT_M_I1_mini_switch.is_closed()</code>	True / False
Fototransistor an I3, LED an 9 V	Fototransistor I3	<code>TXT_M_I3_photo_transistor.is_dark()</code>	True = Lichtschanke unterbrochen
IR-Spursensor an I1/I2, 9 V	Spursensor	<code>TXT_M_I1_trail_follower.get_state()</code>	1 = weiß, 0 = schwarz
Ultraschall an I2, 9 V	Ultraschall	<code>TXT_M_I2_ultrasonic_distance_meter.get_distance()</code>	Abstand in cm
LED an O3	LED O3	<code>TXT_M_O3_led.set_brightness(512)</code>	Helligkeit 0 ... 512
Encodermotor an M1, Encoder an C1	Encodermotor M1 + Zähler C1	<code>TXT_M_M1_encodermotor.set_speed(300, Motor.CCW)</code>	Drehrichtung CW / CCW

3.2 if, elif, else

```
abstand = TXT_M_I2_ultrasonic_distance_meter.get_distance()
if abstand < 15:                # Bedingung + Doppelpunkt
    print("STOPP")              # eingerückt (4 Leerzeichen) = gehört dazu
elif abstand < 40:
    print("langsam")
else:
    print("freie Fahrt")
```

Vergleich	Bedeutung	Logik	Bedeutung (LB 2)
<code>== / !=</code>	gleich / ungleich	<code>and</code>	UND – Reihenschaltung
<code>< > <= >=</code>	kleiner / größer (gleich)	<code>or</code>	ODER – Parallelschaltung
		<code>not</code>	NICHT – Umkehrung

Häufige Fehler

Doppelpunkt vergessen · falsche Einrückung · = statt == im Vergleich · Grenzwerte in falscher Reihenfolge geprüft (zuerst den kleinsten!).

Ausprobieren im Browser

Lernseite **L03 Entscheidungen** (Ordner 05_HTML_Lernseiten). Aufgaben: UND/ODER-Tabelle, Taster schaltet LED, Abstandswarner, Schranke schließen bis zur Endlage.

Teste dich

1. Was liefert `not TXT_M_I1_mini_switch.is_closed()`, wenn der Taster gedrückt ist?
2. Schreibe eine Bedingung: Abstand kleiner 20 cm UND Taster gedrückt.
3. Welcher Befehl liest einen IR-Spursensor an I2?

4 Schleifen und Zustandsautomaten

Lernziele dieses Kapitels

- for- und while-Schleifen passend auswählen
- ein Zustandsübergangsdiagramm in Code übersetzen
- Zähler und Encoder-Impulse verwenden (Fischer A2)

4.1 Schleifen

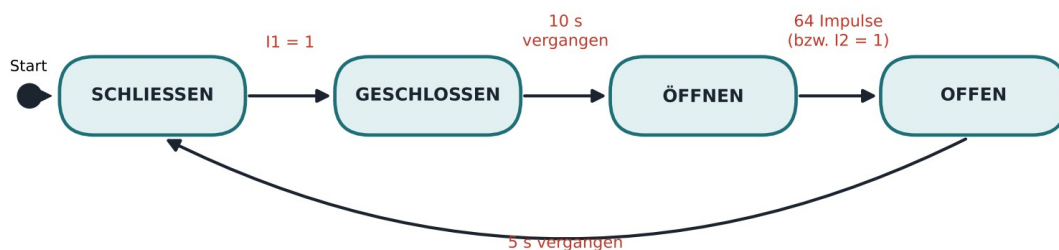
```
for i in range(3):                                # genau 3-mal (i = 0, 1, 2)
    print("Zyklus", i + 1)

while not TXT_M_I1_mini_switch.is_closed():       # solange Bedingung gilt
    time.sleep(0.01)                               # kurz warten, CPU schonen

while True:                                       # endlos – Ausstieg mit break
    if TXT_M_I2_mini_switch.is_closed():
        break
```

4.2 Zustandsautomaten

Ein **Zustandsautomat** (endlicher Automat) befindet sich immer in genau einem Zustand. Ereignisse (Taster, Zeit, Impulse) lösen **Übergänge** aus. Zuerst wird das Diagramm gezeichnet, dann jeder Zustand ein Zweig einer **if/elif**-Kette.



```
zustand = "SCHLIESSEN"                            # unbekannte Startlage -> zuerst schließen
while True:
    if zustand == "SCHLIESSEN":
        schliessen()                               # Motor CW bis Endlagentaster I1
        zustand = "GESCHLOSSEN"
    elif zustand == "GESCHLOSSEN":
        time.sleep(10)
        zustand = "OEFFNEN"
    elif zustand == "OEFFNEN":
        TXT_M_M1_encodermotor.set_speed(300, Motor.CCW)
        TXT_M_M1_encodermotor.set_distance(64)      # 64 Impulse = 90°
        while TXT_M_M1_encodermotor.is_running():
            time.sleep(0.01)
        zustand = "OFFEN"
    elif zustand == "OFFEN":
        time.sleep(5)
        zustand = "SCHLIESSEN"
```

Zähler

Ein Zähler ist eine Variable, die bei jedem Ereignis erhöht wird: `durchfahrten = durchfahrten + 1` (kurz: `durchfahrten += 1`). Encoder-Zähler liest man mit `TXT_M_C1_motor_step_counter.get_count()` und setzt sie mit `reset()` zurück.

Ausprobieren im Browser

Lernseite **L04 Schleifen, Zähler und Zustände** (Ordner 05_HTML_Lernseiten). Aufgaben: Countdown, automatische Schranke, Öffnen mit Encoder, Bedarfstaster + Lichtschranke + Zähler.

Am echten TXT 4.0

Fischer-Aufgabe A2 „Parkhausschranke“ – zuerst in Blockly (DS 3, Arbeitsblatt 3), dann in Python (DS 4, Arbeitsblatt 4).

Teste dich

1. Welche Schleife wählst du für „warte, bis die Lichtschranke frei ist“?
2. Wie viele Durchläufe hat `for i in range(2, 10, 2):`?
3. Warum braucht ein Zustandsautomat einen definierten Startzustand?

5 Funktionen, Zeitsteuerung und Threads

Lernziele dieses Kapitels

- Funktionen mit Parametern und Rückgabewert schreiben
- zeitgesteuerte Abläufe programmieren
- nebenläufige Prozesse erklären (Fischer A3)

5.1 Funktionen

```
def rot():                                # ohne Parameter
    TXT_M_03_led.set_brightness(512)
    TXT_M_04_led.set_brightness(0)

def blinken(anzahl, dauer):              # mit Parametern
    for i in range(anzahl):
        TXT_M_01_led.set_brightness(512); time.sleep(dauer)
        TXT_M_01_led.set_brightness(0);   time.sleep(dauer)

def impulse_fuer_winkel(winkel):         # mit Rückgabewert
    return round(winkel / 90 * 64)

rot()
blinken(5, 0.5)
print(impulse_fuer_winkel(45))          # 32
```

Funktionen entsprechen den Unterprogrammen („definiere ...“) in Blockly. Sie machen Programme **kürzer, lesbarer und wiederverwendbar**. Variablen, die in einer Funktion verändert werden sollen, aber außerhalb definiert sind, kennzeichnet man mit `global`.

5.2 Zwei Dinge gleichzeitig

Solange `time.sleep()` läuft, tut das Programm nichts anderes. Soll ein Warnlicht blinken, **während** die Schranke fährt, gibt es zwei Wege:

```
# Weg 1: Thread (nebenläufiger Prozess) – auf dem TXT
import threading
blinken_aktiv = False
def blinklicht():
    while True:
        if blinken_aktiv:
            TXT_M_01_led.set_brightness(512); time.sleep(0.5)
            TXT_M_01_led.set_brightness(0);   time.sleep(0.5)
        else:
            time.sleep(0.05)
threading.Thread(target=blinklicht, daemon=True).start()

# Weg 2: Zeitscheiben – nie lange warten, sondern auf die Uhr schauen
letztes = time.time()
while TXT_M_M1_encodermotor.is_running():
    if time.time() - letztes >= 0.5:
        umschalten()
        letztes = time.time()
    time.sleep(0.01)
```

Ausprobieren im Browser

Lernseite **L05 Funktionen und Zeitsteuerung** (Ordner 05_HTML_Lernseiten). Aufgaben: Blinklicht, Bedarfsampel, Funktion mit Rückgabewert, Blinken während die Schranke öffnet.

Am echten TXT 4.0

Fischer-Aufgabe A3 „Ampel und Blinklicht“ – erst Blockly, dann Python (DS 5). Vorlage: 06_TXT_Programme/A3_Ampel_Blinklicht_Thread.py.

Teste dich

1. Was ist der Unterschied zwischen Parameter und Rückgabewert?
2. Was gibt `impulse_fuer_winkel(180)` zurück?
3. Warum blinkt ein Blinklicht in einem Thread gleichmäßig?

6 Binärcode und Listen

Lernziele dieses Kapitels

- Dualzahlen in Dezimalzahlen umrechnen und umgekehrt
- Listen anlegen und auswerten
- einen Barcode dekodieren und Fehler erkennen (Fischer A4)

6.1 Das Dualsystem

Computer kennen nur zwei Zustände: 0 und 1 (**Bit**). 8 Bit = 1 **Byte**. Jede Stelle einer Dualzahl ist doppelt so viel wert wie die rechts daneben.

128	64	32	16	8	4	2	1	= dezimal
0	0	1	0	1	1	0	1	$32 + 8 + 4 + 1 = 45$

```
print(bin(45))          # '0b101101'
print(int("101101", 2)) # 45

def dual_zu_dezimal(text):      # selbst programmiert
    wert = 0
    for ziffer in text:
        wert = wert * 2 + int(ziffer)
    return wert
```

6.2 Listen

Befehl	Bedeutung
<code>breiten = [22, 8, 21]</code>	Liste anlegen
<code>breiten[0], breiten[-1]</code>	erstes / letztes Element (Index beginnt bei 0!)
<code>len(), min(), max(), sum()</code>	Anzahl, kleinster, größter Wert, Summe
<code>breiten.append(9)</code>	Element anhängen
<code>codes.index("100001001")</code>	Position eines Elements
<code>for b in breiten:</code>	alle Elemente durchlaufen

6.3 Der Barcode Code 39

Ein Zeichen besteht aus 9 Elementen (5 Balken, 4 Lücken), schmal = 0, breit = 1. Gültige Zeichen haben **genau drei** breite Elemente. Von $2^9 = 512$ möglichen Mustern werden nur 43 genutzt – diese **Redundanz** ermöglicht die Fehlererkennung.

```
def dekodieren(breiten):
    schwelle = (min(breiten) + max(breiten)) / 2
    bits = ""
    for b in breiten:
        bits += "1" if b > schwelle else "0"
    if bits.count("1") != 3 or bits not in codes:
        return "Fehler"
    return zeichen[codes.index(bits)]      # zwei Listen: zeichen / codes
```

Ausprobieren im Browser

Lernseite **L06 Binärbarcode und Listen** (Ordner 05_HTML_Lernseiten). Aufgaben: Dual → Dezimal, Messreihe auswerten, Balken → Bits, Code-39-Dekodierer.

Am echten TXT 4.0

Fischer-Aufgabe A4 „Kodierung“ mit Kamera (Linienerkennung). Vorlage: 06_TXT_Programme/A4_Barcode_Code39.py.

Teste dich

1. Rechne 1100100 in eine Dezimalzahl um.
2. Was liefert `[3, 6, 9][-1]`?
3. Warum kann Code 39 Lesefehler erkennen?

7 Texte, Wörterbücher und Verschlüsselung

Lernziele dieses Kapitels

- Texte zeichenweise verarbeiten
- Wörterbücher als Übersetzungstabellen einsetzen
- eine Nachricht kodieren, senden und verschlüsseln (Fischer A5)

7.1 Strings und Dictionaries

```
text = "Robotik"
print(len(text), text[0], text.upper())    # 7 R ROBOTIK
for z in text:                             # jedes Zeichen
    print(z)

morse = {"S": "...", "O": "---", "T": "-"} # Schlüssel -> Wert
print(morse["O"])                         # ---
rueck = {w: s for s, w in morse.items()}   # umgekehrtes Wörterbuch
```

7.2 Morse-Code über eine Lichtstrecke

Ein **Kommunikationskanal** besteht aus Sender (LED), Übertragungsweg (Licht) und Empfänger (Fototransistor). Damit Sender und Empfänger sich verstehen, brauchen sie einen gemeinsamen **Code** und einen **Takt**: dit = 1 Einheit, dah = 3 Einheiten, Pause im Zeichen = 1, zwischen Buchstaben = 3, zwischen Wörtern = 7.

```
DIT = 0.1
def signal(dauer):
    TXT_M_01_led.set_brightness(512); time.sleep(dauer)
    TXT_M_01_led.set_brightness(0);    time.sleep(DIT)

def sende(text):
    for buchstabe in text.upper():
        for z in morse[buchstabe]:
            signal(DIT if z == "." else 3 * DIT)
            time.sleep(2 * DIT)
```

7.3 Verschlüsselung

Die **Caesar-Chiffre** verschiebt jeden Buchstaben um k Stellen. Mit `ord()` (Buchstabe → Zahl) und `chr()` (Zahl → Buchstabe) und Modulo 26 bleibt man im Alphabet. Sie ist leicht zu knacken (nur 25 Schlüssel). Die **Vigenère-Chiffre** verwendet ein Schlüsselwort und ist deutlich sicherer.

```
def caesar(text, k):
    return "".join(chr((ord(b) - 65 + k) % 26 + 65) for b in text.upper())
print(caesar("ROBOTIK", 3))    # URERWLN
```

Ausprobieren im Browser

Lernseite **L07 Texte und Wörterbücher** (Ordner 05_HTML_Lernseiten). Aufgaben: Text → Morse, Morse-Sender mit LED, Empfänger dekodieren, Caesar.

Am echten TXT 4.0

Fischer-Aufgabe A5 „Morse-Code“ – am besten zu zweit mit zwei TXT testen. Vorlage: 06_TXT_Programme/A5_Morse_Sender.py.

Teste dich

1. Was liefert `"TXT"[1]`?
2. Wie lange dauert „E“ (ein dit) inklusive Buchstabenpause bei $DIT = 0,1 \text{ s}$?
3. Entschlüssele „URERW“ ($k = 3$).

8 Motoren, Encoder und Kalibrierung

Lernziele dieses Kapitels

- Motoren zeit- und impulsgesteuert ansteuern
- einen Roboter kalibrieren
- eine Bremsrampe berechnen (Fischer A6, Material 3.1/3.2)

8.1 Motoren ansteuern

Befehl	Wirkung
<code>m.set_speed(400, Motor.CCW)</code>	Geschwindigkeit 0 ... 512 und Drehrichtung (beim Buggy: CCW = vorwärts)
<code>m.start() / m.stop()</code>	Motor ein / aus
<code>m1.start_sync(m2)</code>	beide Motoren synchron (m2 folgt m1)
<code>m1.set_distance(300, m2)</code>	genau 300 Impulse fahren (synchron)
<code>m.is_running()</code>	True, solange der Motor noch fährt
<code>TXT_M_C1_motor_step_counter.get_count()</code>	Impulse seit dem letzten <code>reset()</code>

Zwei Motoren sind nie völlig gleich – ohne Synchronisation fährt der Buggy eine Kurve. Bei `start_sync` vergleicht der TXT ständig die Encoder-Impulse und regelt nach (Master-Slave-Prinzip).

8.2 Kalibrieren

Der Encoder liefert 63,9 Impulse je Motorumdrehung. Wie viele Impulse einem Zentimeter entsprechen, bestimmt man durch **Messen**: eine feste Impulszahl fahren, Strecke messen, teilen – am besten mehrmals und den Mittelwert bilden. Ebenso für Drehungen auf der Stelle (Impulse pro Grad).

Messgröße	Rechnung	Beispielwert
Impulse pro cm	$\text{Impulse} \div \text{gemessene Strecke}$	$500 \div 77,7 \text{ cm} \approx 6,44$
Impulse pro Grad	$\text{Impulse für } 360^\circ \div 360$	$\approx 0,751$
Impulse für 2 m	$200 \cdot 6,44$	≈ 1288

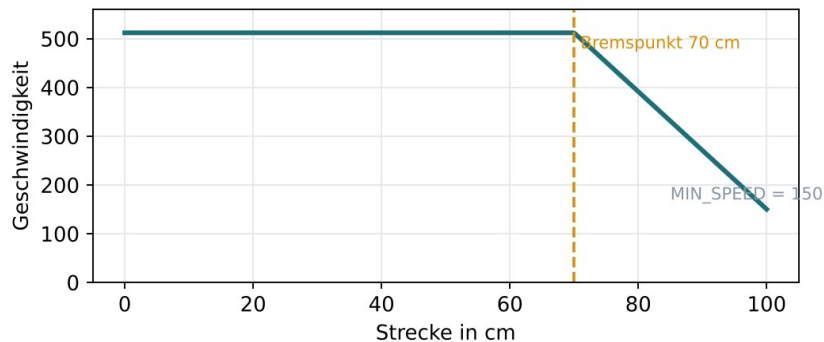
```
IMPULSE_PRO_CM, IMPULSE_PRO_GRAD = 6.438, 0.751
L, R = TXT_M_M1_encodermotor, TXT_M_M2_encodermotor

def bewege(impulse, richtung_links, richtung_rechts, speed=400):
    L.set_speed(speed, richtung_links)
    R.set_speed(speed, richtung_rechts)
    L.set_distance(round(impulse), R)
    while L.is_running():
        time.sleep(0.01)

def vor(cm):    bewege(cm * IMPULSE_PRO_CM, Motor.CCW, Motor.CCW)
def rechts(grad): bewege(grad * IMPULSE_PRO_GRAD, Motor.CCW, Motor.CW)
def links(grad): bewege(grad * IMPULSE_PRO_GRAD, Motor.CW, Motor.CCW)
```

8.3 Die Bremsrampe (Material 3.2)

Statt abrupt zu stoppen, wird die Geschwindigkeit in der Bremszone **linear** abgesenkt: $\text{Fortschritt} = (\text{Ticks} - \text{Ticks_Brems}) \div (\text{Ticks_Ziel} - \text{Ticks_Brems})$ liegt zwischen 0 und 1, $\text{Geschwindigkeit} = \text{MAX} - \text{Fortschritt} \cdot (\text{MAX} - \text{MIN})$. MIN bleibt über 0, damit die Haftreibung überwunden wird.



Ausprobieren im Browser

Lernseite **L08 Fahren mit Encodern** (Ordner 05_HTML_Lernseiten). Aufgaben: vor–Pause–zurück, Kalibrierfahrt, fahre_cm/drehe, Bremsrampe.

Am echten TXT 4.0

Fischer-Aufgabe A6 „Buggy“, Programmieraufgaben 1–5. Kalibrierwerte des eigenen Buggys auf Arbeitsblatt 8 notieren!

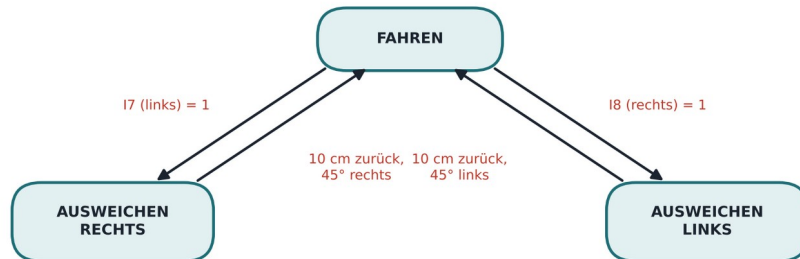
Teste dich

1. Wie viele Impulse sind 35 cm (6,44 Impulse/cm)?
2. Warum ist Fahren nach Impulsen genauer als nach Zeit?
3. Welche Geschwindigkeit ergibt die Rampe bei Fortschritt 0,5 (MAX 400, MIN 100)?

9 Sensorgesteuertes Fahren: Hindernisse

Lernziele dieses Kapitels

- Hindernisse mit Taster, Ultraschall und IR-Sensoren erkennen
- ein Ausweichmanöver als Unterprogramm schreiben
- einen Zustandsautomaten für den Buggy entwerfen (Fischer A6, Material 3.3)



```
zustand = "FAHREN"
while True:
    if zustand == "FAHREN":
        geradeaus()
        if TXT_M_I7_mini_switch.is_closed():          # Stoßstange links
            zustand = "AUSWEICHEN_RECHTS"
        elif TXT_M_I8_mini_switch.is_closed():        # Stoßstange rechts
            zustand = "AUSWEICHEN_LINKS"
    elif zustand == "AUSWEICHEN_RECHTS":
        zurueck(10); rechts(45); zustand = "FAHREN"
    elif zustand == "AUSWEICHEN_LINKS":
        zurueck(10); links(45); zustand = "FAHREN"
    time.sleep(0.02)
```

9.1 Ultraschall

Der Sensor sendet einen Ultraschall-Ping und misst die Zeit t bis zum Echo: **Abstand** = $t \cdot 343 \text{ m/s} \div 2$ (der Schall läuft hin und zurück). Material 3.3: Ist der Abstand $\leq 20 \text{ cm}$, umfährt der Buggy das Hindernis als „halbes Rechteck“ (90° rechts, 40 cm, 90° links, 40 cm, 90° links, 40 cm, 90° rechts). Die Kantenlänge steht in einer Variablen `distance` – keine „Magic Numbers“ im Code!

9.2 IR-Spursensoren

Die beiden IR-Sensoren liefern 1 auf weißem Untergrund und 0 auf einer schwarzen Linie oder an einer Tischkante. Damit bleibt der Buggy in einem Bereich (A6, Aufgabe 7) oder folgt einer Spur (Kapitel 12).

Ausprobieren im Browser

Lernseite **L09 Hindernisse erkennen** (Ordner 05_HTML_Lernseiten). Aufgaben: Stoßstangen-Buggy, Hindernis umfahren mit Ultraschall, im Kreis bleiben.

Teste dich

1. Das Echo kommt nach 2,9 ms. Wie weit ist das Hindernis entfernt?
2. Warum steht die Kantenlänge des Ausweichmanövers in einer Variablen?
3. Welche Werte liefern beide IR-Sensoren an einer Tischkante?

10 Kartesische Koordinaten: der Malroboter

Lernziele dieses Kapitels

- geometrische Figuren mit Funktionen und Schleifen zeichnen
- Abstand und Richtung zwischen Punkten berechnen
- einen Roboter Koordinaten anfahren lassen (Fischer A7)

10.1 n-Ecke

Für ein regelmäßiges n-Eck fährt der Roboter n-mal eine Kante und dreht an jeder Ecke um den **Außenwinkel $360^\circ \div n$** .

```
def n_eck(n, laenge):
    for i in range(n):
        vor(laenge)
        links(360 / n)

n_eck(6, 20)      # Sechseck, Kante 20 cm
```

10.2 Rechnen im Koordinatensystem

Im kartesischen Koordinatensystem beschreibt ein Zahlenpaar $(x \mid y)$ jeden Punkt. Von $A(x_1 \mid y_1)$ nach $B(x_2 \mid y_2)$:

- **Abstand** (Pythagoras): $d = \sqrt{(\Delta x^2 + \Delta y^2)}$ mit $\Delta x = x_2 - x_1$, $\Delta y = y_2 - y_1$
- **Richtung**: $\alpha = \text{atan2}(\Delta y, \Delta x)$ – liefert den Winkel in allen vier Quadranten ($-180^\circ \dots 180^\circ$)
- **Drehung** = Zielrichtung – aktuelle Blickrichtung, auf $-180^\circ \dots 180^\circ$ gebracht

```
pos_x, pos_y, blick = 0.0, 0.0, 0.0

def fahre_zu(zx, zy):
    global pos_x, pos_y, blick
    dx, dy = zx - pos_x, zy - pos_y
    ziel = math.degrees(math.atan2(dy, dx))
    drehung = (ziel - blick + 180) % 360 - 180
    if drehung > 0:
        links(drehung)
    else:
        rechts(-drehung)
    vor(math.hypot(dx, dy))
    pos_x, pos_y, blick = zx, zy, ziel

for x, y in [(20, 0), (20, 20), (0, 20), (0, 0)]: # "Malen nach Zahlen"
    fahre_zu(x, y)
```

Kalibrierung + Koordinaten

Aus Δx und Δy werden Strecke und Winkel, über die Kalibrierwerte daraus Impulse. Rundungs- und Schlupffehler addieren sich – deshalb nutzen echte Roboter zusätzlich Sensoren zur Positionskorrektur (z. B. Kamera, Linien, Funkbaken).

Ausprobieren im Browser

Lernseite **L10 Kartesische Koordinaten** (Ordner 05_HTML_Lernseiten). Aufgaben: n-Eck, Haus vom Nikolaus, Abstand und Richtung, Zielpunkte anfahren, Malen nach Zahlen.

Am echten TXT 4.0

Fischer-Aufgabe A7 „Malroboter“ inkl. Experimentieraufgaben 1–3 (DS 10 und 11, Arbeitsblätter 10 und 11).

Teste dich

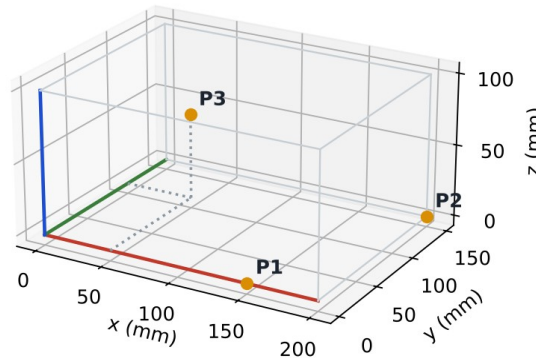
1. Außenwinkel eines Achtecks?
2. Abstand und Richtung von $(0 \mid 0)$ nach $(-30 \mid 0)$?
3. Der Roboter blickt nach 90° und muss nach -120° . Um wie viel und in welche Richtung dreht er?

11 3D-Koordinaten

Lernziele dieses Kapitels

- Punkte im Raum mit (x | y | z) angeben
- Abstände im Raum berechnen
- Achsen kalibrieren und Bewegungen mit Sicherheitshöhe planen

Industrieroboter, 3D-Drucker, CNC-Fräsen und Portalroboter im Lager arbeiten im **dreidimensionalen** kartesischen Koordinatensystem. Zur x- und y-Achse kommt die z-Achse (Höhe).



- Punkt im Raum: P(x | y | z), z. B. P3(50 | 100 | 60) in mm
- Abstand: $d = \sqrt{(\Delta x^2 + \Delta y^2 + \Delta z^2)}$ – Pythagoras zweimal angewendet
- Raumdiagonale eines Quaders $a \times b \times c$: $\sqrt{a^2 + b^2 + c^2}$
- Jede Achse wird einzeln kalibriert (Impulse pro mm), weil Spindeln, Zahnriemen und Getriebe verschieden sind.

11.1 Der Portalroboter in Python

```
TICKS_PRO_MM = [2.0, 2.0, 4.0] # x, y, z (kalibriert)
ACHSEN = [TXT_M_M1_encodermotor, TXT_M_M2_encodermotor, TXT_M_M3_encodermotor]
pos = [0.0, 0.0, 100.0]
SICHER = 60 # Sicherheitshöhe in mm

def bewege(ziel): # alle Achsen gleichzeitig
    for i in range(3):
        mm = ziel[i] - pos[i]
        ticks = round(abs(mm) * TICKS_PRO_MM[i])
        if ticks > 0:
            ACHSEN[i].set_speed(400, Motor.CCW if mm > 0 else Motor.CW)
            ACHSEN[i].set_distance(ticks)
    while any(a.is_running() for a in ACHSEN):
        time.sleep(0.01)
    for i in range(3):
        pos[i] = ziel[i]

def fahre_zu(x, y, z): # hoch, waagerecht, runter
    if pos[2] < SICHER:
        bewege([pos[0], pos[1], SICHER])
    bewege([x, y, max(pos[2], SICHER)])
    bewege([x, y, z])
```

Pick & Place

Sicherheitshöhe → über das Teil → absenken → greifen → anheben → über das Ziel → absenken → ablegen → anheben. So stößt der Greifer unterwegs nichts um. Beim 3D-Drucker druckt die Düse nur in der aktuellen Schicht ($z = \text{konstant}$).

Ausprobieren im Browser

Lernseite **L11 3D-Koordinaten** (Ordner 05_HTML_Lernseiten). Aufgaben: Abstand im Raum, z-Achse kalibrieren, fahre_zu mit Sicherheitshöhe, Pick & Place, eine Schicht drucken.

Am echten TXT 4.0

Ein Portal lässt sich mit den Encodermotoren selbst konstruieren (Verbindung zu LB 4 Konstruktion/CAD). Der Roboterarm mit Servos (LB 5) arbeitet mit Gelenkwinkeln; die Umrechnung von $(x \mid y \mid z)$ in Winkel heißt inverse Kinematik.

Teste dich

1. Abstand von $(0 \mid 0 \mid 0)$ nach $(2 \mid 3 \mid 6)$?
2. 120 Impulse = 30 mm. Wie viele Impulse für 45 mm?
3. Warum fährt der Greifer erst auf Sicherheitshöhe?

12 Spurfolger und Regelung

Lernziele dieses Kapitels

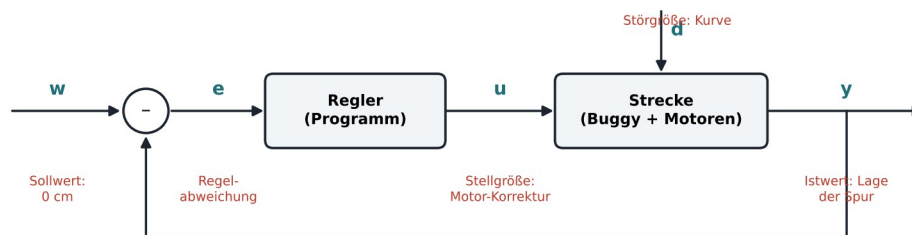
- eine Fallunterscheidung für zwei Sensoren aufstellen
- einen Zweipunkt- und einen P-Regler programmieren
- die Größen eines Regelkreises benennen (Fischer A8)

I1 links	I2 rechts	Bedeutung	Reaktion
0	0	beide auf der Linie	geradeaus
1	0	Linie liegt rechts	rechts lenken (links schnell)
0	1	Linie liegt links	links lenken (rechts schnell)
1	1	Linie verloren	in letzter Richtung suchen

```
def motoren(links, rechts):
    TXT_M_M1_encodermotor.set_speed(int(links), Motor.CCW)
    TXT_M_M2_encodermotor.set_speed(int(rechts), Motor.CCW)
    TXT_M_M1_encodermotor.start(); TXT_M_M2_encodermotor.start()

while True:
    l = TXT_M_I1_trail_follower.get_state()
    r = TXT_M_I2_trail_follower.get_state()
    if l == 0 and r == 0: motoren(300, 300)
    elif l == 1 and r == 0: motoren(300, 60)
    elif l == 0 and r == 1: motoren(60, 300)
    time.sleep(0.01)
```

12.1 Der Regelkreis



Der Zweipunktregler kennt nur „zu weit links/rechts“ – der Buggy zappelt. Mit der Kamera lässt sich die **Abweichung e** (in cm) messen. Der **P-Regler** lenkt proportional: Korrektur = $k_p \cdot e$; links = BASIS – $k_p \cdot e$, rechts = BASIS + $k_p \cdot e$. Zu kleines k_p : Spur geht in Kurven verloren; zu großes k_p : Schwingen. Ein D-Anteil (PD-Regler) reagiert auf die Änderung von e und dämpft das Schwingen.

Ausprobieren im Browser

Lernseite **L12 Spurfolger und Regler** (Ordner 05_HTML_Lernseiten). Aufgaben: Zweipunkt-Spurfolger, P-Regler einstellen.

Am echten TXT 4.0

Fischer-Aufgabe A8 „Spurfolger“: Programmieraufgaben 1–4, Experimentieraufgaben P- und PD-Regler mit Kamera.

Teste dich

1. Was bedeutet $I_1 = 1$ und $I_2 = 0$?
2. BASIS 300, $k_p 40$, $e = 2$ cm: Geschwindigkeiten links/rechts?
3. Nenne Soll- und Istwert beim Spurfolger.

13 Künstliche Intelligenz I: Maschinelles Lernen

Lernziele dieses Kapitels

- Regeln programmieren und aus Daten lernen unterscheiden
- den k-Nächste-Nachbarn-Algorithmus anwenden
- Trainings- und Testdaten nutzen und die Genauigkeit bestimmen

Bisher haben wir dem Roboter **Regeln** vorgegeben („wenn Abstand < 20, dann ausweichen“). Beim **maschinellen Lernen** zeigt man stattdessen **Beispiele mit der richtigen Antwort** (Trainingsdaten). Das Programm leitet die Entscheidung daraus ab. Das ist vor allem dann hilfreich, wenn Regeln schwer zu formulieren sind – etwa bei Farben unter wechselndem Licht.

Begriff	Bedeutung	Beispiel Farberkennung
Merkmal (Feature)	Messgröße, die das Objekt beschreibt	R, G, B der Kamera
Klasse (Label)	richtige Antwort	„rot“, „grün“, „blau“
Trainingsdaten	Beispiele zum Lernen	je Farbe 5–10 Messungen
Testdaten	neue Beispiele zum Prüfen	Messungen an anderer Stelle
Genauigkeit	richtig ÷ alle · 100 %	7 von 8 = 87,5 %

13.1 k-Nächste-Nachbarn

Eine Farbe ist ein Punkt im **3D-Farbraum** (R | G | B). Ähnliche Farben liegen nah beieinander. kNN berechnet den Abstand (wie in Kapitel 11!) zu allen Trainingspunkten und wählt die Klasse der k nächsten Nachbarn (Mehrheit).

```
trainingsdaten = [( [205, 45, 50], "rot"), ([55, 165, 70], "gruen"), ([45, 70, 195], "blau"), ...]

def farbabstand(a, b):
    return math.sqrt((a[0]-b[0])**2 + (a[1]-b[1])**2 + (a[2]-b[2])**2)

def klassifiziere(rgb):
    bester, klasse = 10000, "?"
    for merkmale, label in trainingsdaten:
        d = farbabstand(rgb, merkmale)
        if d < bester:
            bester, klasse = d, label
    return klasse
```

KI im Roboter

Der Buggy liest während der Fahrt RGB-Werte, klassifiziert sie mit kNN und reagiert (rot: anhalten, blau: Ziel). Der Code ist reines Python und läuft unverändert auf dem TXT. Wichtig: mit **eigenen** Messdaten trainieren!

Ausprobieren im Browser

Lernseite **L13 KI 1 – Farben lernen** (Ordner 05_HTML_Lernseiten). Aufgaben: Farbabstand, nächster Nachbar, Genauigkeit, der Buggy reagiert auf Farben.

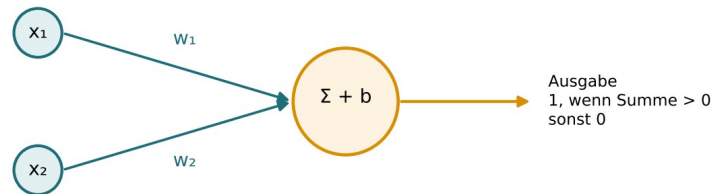
Teste dich

1. Was sind beim Farb-Beispiel Merkmale und Klassen?
2. Warum braucht man Testdaten, die nicht im Training vorkommen?
3. Wie kann man ein kNN-Modell verbessern?

14 Künstliche Intelligenz II: Neuronale Netze und Bilderkennung

Lernziele dieses Kapitels

- den Aufbau eines künstlichen Neurons beschreiben
- die Lernregel des Perzeptrons anwenden
- erklären, wie ein Roboter Bilder erkennt



Ein künstliches **Neuron** multipliziert jeden Eingang mit einem **Gewicht**, addiert alles plus **Bias** b und gibt 1 aus, wenn die Summe größer als 0 ist. Mit $w_1 = w_2 = 1$ und $b = -1,5$ verhält es sich wie ein UND-Gatter, mit $b = -0,5$ wie ODER (vgl. LB 2).

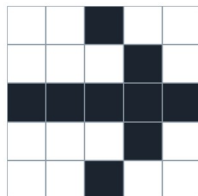
14.1 Lernen

Perzeptron-Lernregel: Für jedes Trainingsbeispiel Fehler = Soll – Ist; dann $w = w + \text{Lernrate} \cdot \text{Fehler} \cdot x$ und $b = b + \text{Lernrate} \cdot \text{Fehler}$. Das wiederholt man mehrere Durchläufe (**Epochen**), bis alle Beispiele stimmen.

```
for epoche in range(20):
    for x, soll in daten:
        fehler = soll - ausgabe(x)
        for i in range(len(w)):
            w[i] = w[i] + lernrate * fehler * x[i]
        b = b + lernrate * fehler
```

14.2 Bilderkennung

Ein Kamerabild ist eine Tabelle aus Pixeln. Ein 5×5-Schwarzweißbild ergibt 25 Eingänge – jedes Pixel bekommt ein eigenes Gewicht. Nach dem Training mit Beispielpfeilen erkennt das Perzeptron, ob ein Schild nach links oder rechts zeigt, und der Buggy biegt entsprechend ab. Moderne Bilderkennung nutzt **neuronale Netze** mit vielen Schichten und Millionen Gewichten. Erst mehrere Schichten können auch Probleme wie XOR lösen.



Nachdenken

KI-Modelle sind nur so gut wie ihre Trainingsdaten. Sie können sich irren und Vorurteile aus den Daten übernehmen. Beim Einsatz von KI in Robotern (Logistik, Verkehr, Pflege) müssen Menschen entscheiden und Verantwortung tragen.

Ausprobieren im Browser

Lernseite **L14 KI 2 – Neuronen und Bilderkennung** (Ordner 05_HTML_Lernseiten). Aufgaben: Neuron als UND, ODER lernen, Pfeile erkennen, der Buggy liest Schilder, XOR-Netz.

Teste dich

1. Wie viele Gewichte hat ein Perzeptron für ein 8×8-Bild (ohne Bias)?
2. $w = [0, 2; 0]$, $b = 0$, $x = (0 \mid 1)$, Soll 1, Ist 0, Lernrate 0,1: neue Werte?
3. Warum kann ein einzelnes Perzeptron XOR nicht lernen?

Anhang A Python-Spickzettel

Thema	Syntax / Beispiel
Ausgabe	<code>print("Wert:", x) · print(f"{x:.2f}")</code>
Variable / Typen	<code>x = 5 · int, float, str, bool · type(x)</code>
Umwandeln	<code>str(x), int(t), float(t), round(x, 1)</code>
Bedingung	<code>if a < 5: ... elif a < 9: ... else:</code>
Logik	<code>and, or, not · Vergleich == != < > <= >=</code>
Zählschleife	<code>for i in range(start, ende, schritt):</code>
Bedingte Schleife	<code>while bedingung: · while True: + break</code>
Funktion	<code>def name(p1, p2): ... return ergebnis</code>
Liste	<code>l = [1, 2] · l[0] · l.append(3) · len(l) · for e in l:</code>
Wörterbuch	<code>d = {"A": "-"} · d["A"] · for k, v in d.items():</code>
Text	<code>t.upper() · t[0] · len(t) · "x" in t · " ".join(liste)</code>
Mathematik	<code>import math · math.sqrt, math.hypot, math.atan2, math.degrees</code>
Zeit	<code>import time · time.sleep(0.5) · time.time()</code>
Thread	<code>threading.Thread(target=funktion, daemon=True).start()</code>

Anhang B TXT-4.0-Befehle (ROBO Pro Coding)

Bauteil	Befehle
Encodermotor M1	<code>set_speed(v, Motor.CW/CCW), start(), stop(), start_sync(m2), stop_sync(m2), set_distance(n, m2), is_running()</code>
Zähler C1	<code>TXT_M_C1_motor_step_counter.get_count(), reset()</code>
LED O1	<code>TXT_M_01_led.set_brightness(0 ... 512)</code>
Mini-Taster I1	<code>is_closed(), is_open()</code>
Fototransistor I3	<code>is_dark(), is_bright()</code>
IR-Spursensor	<code>get_state() → 1 weiß / 0 schwarz</code>
Ultraschall	<code>get_distance() → cm</code>
NTC	<code>get_resistance() → Ω, get_temperature() → °C</code>
Servo S1	<code>set_position(0 ... 512)</code>
Display	<code>from lib.display import * · display.set_attr("label.text", "...")</code>

Die genauen Namen erzeugt ROBO Pro Coding aus der Konfiguration – im Zweifel in der Python-Ansicht nachsehen.

Anhang C Fehlermeldungen verstehen

Meldung	Bedeutung	Typische Ursache
<code>SyntaxError</code>	falsche Schreibweise	Doppelpunkt, Klammer oder Anführungszeichen fehlt
<code>IndentationError</code>	falsche Einrückung	Block nach <code>if/while/def</code> nicht eingerückt
<code>NameError</code>	unbekannter Name	Tippfehler, Groß-/Kleinschreibung, Variable nicht angelegt
<code>TypeError</code>	Typen passen nicht	Text + Zahl ohne <code>str()</code>
<code>IndexError</code>	Index außerhalb der Liste	Zählung beginnt bei 0
<code>AttributeError</code>	Befehl unbekannt	Methodenname falsch geschrieben
Programm hängt	Endlosschleife	keine Abbruchbedingung, kein <code>time.sleep()</code>

Debugging-Strategie

1. Fehlermeldung lesen (Zeilennummer!) · 2. Werte mit `print()` ausgeben · 3. Teile auskommentieren · 4. kleine Schritte testen · 5. Code kommentieren.

Anhang D Lösungen zu „Teste dich“

Kap.	Lösungen
1	Ultraschall = Eingang, LED und Encodermotor = Ausgang · LED am Ausgang O3 des Haupt-Controllers · Code für Menschen lesbar, wird erst in Maschinencode übersetzt
2	int, float, str · 2 · damit schon der erste Messwert kleiner ist und übernommen wird
3	<code>False · abstand < 20 and TXT_M_I1_mini_switch.is_closed() · TXT_M_I2_trail_follower.get_state()</code>
4	<code>while TXT_M_I3_photo_transistor.is_dark():</code> · 4 (2, 4, 6, 8) · die Startlage ist unbekannt; nur so ist das Verhalten eindeutig und sicher
5	Parameter = Eingabewert, Rückgabewert = Ergebnis · 128 · er läuft unabhängig vom Hauptprogramm
6	100 · 9 · nur Muster mit genau 3 breiten Elementen sind gültig (Redundanz)
7	"X" · 0,1 s + 0,3 s = 0,4 s · ROBOT
8	≈ 225 Impulse · unabhängig von Akku und Reibung · 250
9	0,0029 · 343 ÷ 2 ≈ 0,50 m · Änderung an einer Stelle, lesbarer Code · beide 0
10	45° · 30 cm, 180° · -120° - 90° = -210° → +150°, also 150° nach links
11	7 · 180 · damit keine Teile umgestoßen werden (Kollision)
12	Linie liegt rechts · links 220, rechts 380 · Sollwert 0 cm Abweichung, Istwert = gemessene Lage der Spur
13	R, G, B bzw. Farbnamen · um zu prüfen, ob das Modell verallgemeinert · mehr und vielfältigere Trainingsdaten, k anpassen
14	64 · w = [0,2; 0,1], b = 0,1 · die Fälle lassen sich nicht durch eine Gerade trennen